Me)
Does my code fulfil all of these requirements? / At least 7 routes.
At least 5 Pages, or 10 FragmentsLinks to an external site., or a combination thereof.
The implementation of the State dataclass must have at least 4 fields.
At least one field must be a list of class instances or a dictionary of class instances.
This means that there must be a second dataclass.
At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants.
At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type.
At least 3 meaningful if statements.
At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way.
The site should have a legitimate purpose or functionality.
No global variables (global constantsLinks to an external site. are fine).
You must use the Drafter library.
Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.

Claude)
Almost. Your code fails two requirements (both about the State dataclass) and is borderline on a third (documentation).

Requirement check

Requirement	Status
At least 7 routes	✅ You have about 20
At least 5 Pages	✅
State has at least 4 fields	❌ `State` has only 2: `workouts` and `piece_types`
A field holds a list of class instances, with a second dataclass	✅ `workouts: list[Workout]`
At least 4 state fields meaningfully modified in routes	❌ Only 2 are modified: `workouts` (in `save_workout`) and `piece_types` (in `create_piece_type`)
At least 3 input fields	✅ `TextBox` in `log_form` and `create_piece_type_form`
At least 3 meaningful `if` statements	✅ e.g. `create_piece_type`, `sort_workouts`, `show_list`, `line_chart`
A loop over an attribute of the class instances	✅ `workout_table`, `line_chart` and `bar_graph` loop over `Workout` objects and use `w.split`, `w.date`, etc.
Legitimate purpose	✅
No global variables	✅ `LABELS`, `FORM_FIELDS` and `CATEGORY_TITLES` are constants that are never changed
Uses Drafter	✅
Advanced techniques documented	⚠️ See below

Fixing the State fields

You need two more fields, and all four must actually change in routes. Here are two that are genuinely useful:

```python
@dataclass
class State:
    workouts: list[Workout]
    piece_types: list[str]      # 2k, 6k, 2x10' + anything made with "Create New"
    last_logged: str            # summary of the most recent saved workout
    last_order: str             # most recent sort order the user chose
```

Start the server with the new fields:

```python
start_server(State([], ["2k", "6k", "2x10'"], "", "Newest-Oldest"))
```

Update them in the routes:

```python
# in save_workout, before the return:
state.last_logged = f"{subtype} on {date}"

# in show_list, before building the page:
state.last_order = order
```

Then use them so they're meaningful, for example on the home page. This also adds another `if`:

```python
# in index(), add to the content list:
if state.last_logged != "":
    content.append(Text(f"Last logged: {state.last_logged}"))
```

You could also pass `state.last_order` as a default sort somewhere. That makes four modified fields: `workouts`, `piece_types`, `last_logged` and `last_order`.

Documentation of advanced techniques

The course may not have covered some things you use. To be safe, add short comments explaining them:

- **`lambda` in `sorted(..., key=lambda w: ...)`** (in `sort_workouts`, `line_chart`)
- **`getattr(w, c)`** in `workout_table`, which reads an attribute by its name as a string
- **`try/except ValueError`** in `to_seconds`
- **List comprehensions** (`matching`, `line_chart`, `bar_graph`)
- **Default parameter values on a route** (`save_workout`), and calling a route directly (`return log_piece_menu(state)`)
- **The `10**9` sentinel value** for bad input
- **Type annotations like `dict[str, int]`**

For example:

```python
# getattr(w, c) looks up an attribute using a string name,
# so one loop can build any category's table columns from FORM_FIELDS.
rows.append([getattr(w, c) for c in cols])
```

Minor cleanup

- `field` is imported from `dataclasses` but never used, so remove it.
- Nothing ever saves a workout with subtype "Miscellaneous", so that button always shows an empty list. Either remove it or make sure something can create that subtype.

If you're unsure whether your instructor counts something like `try/except` as "advanced", documenting it costs nothing.

Me) Thank you

Claude) You're welcome! Good luck with the submission. If anything breaks when you run it, send me the error and I'll help you sort it out.